

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-28 (44ee8e0+수정중) → 2026-08-29 · 578d95c+수정중 | 2개 파일 · +8 / -7 줄 · 코드 12쪽 | 새 코드북 69쪽

바뀐 내용

- 파워 스펙트럼의 데이터 축을 sa 판과 완전히 맞췄다: 실수부·단축(0..fs/2) welch. 이전엔 해석신호 양측 축이라 왼쪽 절반이 이상한 고원으로 보였는데 그게 사라지고, sa PNG 의 PSD 판과 같은 자리에 같은 모양이 나온다. 고칠 파일은 spectrum.py 와 app.js 둘뿐이다.

□ <code>signus/spectrum.py</code>	+5	-4	2쪽	코드북 9쪽
□ <code>signus/web/app.js</code>	+3	-3	2쪽	코드북 51쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/spectrum.py +5 -4 · 코드북 9쪽 · 새로 쓸 줄 12-16

```

1 1 """Spectrum + spectrogram-strip views (display only, never used for detection)."""
2 2
3 3 import numpy as np
4 4 from scipy.signal import stft, welch
5 5
6 6
7 7 def _db(p: np.ndarray) -> np.ndarray:
8 8     return 10 * np.log10(p + 1e-20)
9 9
10 10
11 11 def spectrum(x: np.ndarray, fs: float, bins: int = 256) -> dict:
12 +     """sa 프로브의 PSD 판과 같은 조판: 실수부 welch(nperseg<=2048), 단축 0..fs/2 (kHz).
13 +     표시 칸으로 줄일 때는 최대 풀링 -- 좁은 바늘이 살아남게."""
14 +     xd = x.real if np.iscomplexobj(x) else x
15 +     f, p = welch(xd, fs=fs, nperseg=int(min(2048, max(64, xd.size))))
16 +     p = _db(p)
17     if f.size > bins: # max-pool so narrow tones survive decimation
18         k = f.size // bins
19         f, p = f[: k * bins].reshape(bins, k).mean(1), p[: k * bins].reshape(bins, k).max(1)
20     return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}
21
22 22
23 23 def strip(x: np.ndarray, fs: float, max_cols: int = 1600) -> dict:
24 24     """sa 프로브의 PNG 스펙트로그램 띠와 같은 조판의 회색조 이미지 (0..235, row-major,
25 25     위 행 = +fs/2 쪽). hamming 256/128 STFT, 바닥 p25 + 대비폭 10..35 dB 클램프, 열린
26 26     최대 풀링으로만 줄인다 -- 보간이 짧은 버스트를 뭉개지 않게."""
27 27     xd = x.real if np.iscomplexobj(x) else x
28 28     nper = int(min(256, max(64, 1 << int(np.log2(max(xd.size // 2, 64)))))
29 29     _, _, z = stft(xd, fs=fs, window="hamming", nperseg=nper, noverlap=nper // 2,
30 30     return_onesided=True, boundary=None, padded=False)
31 31     db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
32 32     lo = float(np.percentile(db, 25))
33 33     rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
34 34     g = np.clip((db - lo) / rg, 0, 1)[::-1]
35 35     kp = max(1, g.shape[1] // max_cols)
36 36     g = g[:, : g.shape[1] // kp * kp].reshape(g.shape[0], -1, kp).max(2)
37 37     return {"rows": int(g.shape[0]), "cols": int(g.shape[1]),
38 38     "g": np.round(g * 235).astype(int).ravel().tolist()}

```

signus/web/app.js +3 -3 · 코드북 51쪽 · 새로 쓸 줄 430, 432, 437

```

1 1 "use strict";
2 2 const $ = (id) => document.getElementById(id);
3 3 const API = "/api/analyze";
4 4 const FS_RE = /(?:^|[_])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
5 5 const RF_RE = /(?:^|[_])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
6 6 const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
7 7 const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
8 8     burst: null, overview: null };
9 9
10 10 /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
11 11 const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
12 12 // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
13 13 const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64" };
14 14 const FMTS = { cplx: "iq", real: "real", iq: "iq" }; // iq = 옛 밀줄 형식의 이름
15 15 /* <이름>.cplx|real.<샘플레이트>.<16t>.pcm - 샘플레이트는 접두사가 없으므로 포맷 바로
16 16     다음 자리로 찾는다. 옛 밀줄 형식(<이름>_fs_<이름>_iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
17 17     같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
18 18 function parseName(name) {

```

```

19 19  const base = name.replace(/^.*\//, "").toLowerCase();
20 20  const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[_./]/);
21 21  // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
22 22  // u8/be 오염과 점 켜 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙)
23 23  const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
24 24  || t === "pcm" || t.startsWith("rf");
25 25  const hit = (k) => /^d+$/i.test(toks[k + 1] || "") && spec(toks[k + 2] || "");
26 26  const cand = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
27 27  const hits = cand.filter(hit);
28 28  const i = hits.length ? hits[hits.length - 1] : (cand.length ? cand[0] : -1);
29 29  const tail = i >= 0 ? toks.slice(i + 1) : toks; // 제원은 포맷 토막 뒤에만 산다
30 30  const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
31 31  return {
32 32  fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
33 33  fmt: i >= 0 ? FMTS[toks[i]] : null,
34 34  dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
35 35  endian: tail.includes("be") ? "be" : "le",
36 36  bitrev: tail.includes("bitrev"),
37 37  rf: rt ? parseFloat(rt[1]) : null,
38 38  };
39 39  }
40 40  /* SigMF: core:datatype -> our fmt/dtype/endian */
41 41  const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
42 42  cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
43 43  function metaFromSigmf(doc) {
44 44  const g = doc && doc.global;
45 45  if (!g || !g["core:datatype"]) return null;
46 46  const dt = g["core:datatype"], base = dt.replace(/[_lb]e$/, "");
47 47  if (!(base in SIGMF)) return null;
48 48  const [fmt, dtype] = SIGMF[base];
49 49  const cap = (doc.captures || [])[0] || {};
50 50  return { fs: Number(g["core:sample_rate"]), fmt, dtype,
51 51  endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
52 52  rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
53 53  }
54 54
55 55  /* --- number formatting --- */
56 56  function sig(x) {
57 57  const a = Math.abs(x);
58 58  return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
59 59  }
60 60  function fmtHz(v) {
61 61  if (v == null) return "-";
62 62  const a = Math.abs(v);
63 63  if (a >= 1e6) return sig(v / 1e6) + " MHz";
64 64  if (a >= 1e3) return sig(v / 1e3) + " kHz";
65 65  return sig(v) + " Hz";
66 66  }
67 67  function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
68 68  return (v / 1e6).toFixed(3) + " MHz";
69 69  }
70 70
71 71  /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
72 72  function idealPoints(mod) {
73 73  if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
74 74  if (mod === "qpsk" || mod === "8psk") {
75 75  const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
76 76  for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
77 77  f + 2 * Math.PI * k / m) });

```

```

77 77     return p;
78 78 }
79 79 const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
80 80 for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
81 81 for (const qi of lv) for (const ii of lv) {
82 82     if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
83 83     p.push({ i: ii, q: qi });
84 84 }
85 85 let e = 0;
86 86 for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
87 87 const norm = Math.sqrt(e / p.length);
88 88 return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
89 89 }
90 90
91 91 /* --- file intake: one data file (+sidecar) or many (batch) --- */
92 92 async function acceptFiles(list) {
93 93     const files = [...list];
94 94     const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
95 95     const data = files.filter((f) => !metas.includes(f));
96 96     if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
97 97     if (data.length > 1) return runBatch(data, metas);
98 98
99 99     state.file = data[0];
100 100     state.resp = null;
101 101     state.batch = [];
102 102     state.burst = null;
103 103     state.overview = null;
104 104     // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
105 105     // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
106 106     const mine = (m, ext) => {
107 107         const stem = m.name.toLowerCase().slice(0, -ext.length);
108 108         const dn = state.file.name.toLowerCase();
109 109         return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
110 110     };
111 111     const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-m
    eta"));
112 112     const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
113 113     state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
114 114     state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
115 115     if (state.meta && state.meta.fs && state.meta.fmt) runFirst();
116 116     else showMetaForm();
117 117 }
118 118 const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
119 119
120 120 function showMetaForm() {
121 121     hideAll();
122 122     state.meta = state.meta || {};
123 123     if (state.meta.fs) $("mFs").value = state.meta.fs;
124 124     $("mFmt").value = state.meta.fmt || "";
125 125     $("mDtype").value = state.meta.dtype || "i16";
126 126     show($("metaForm"));
127 127 }
128 128 $("metaGo").onclick = () => {
129 129     const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
130 130     if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
131 131     state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
132 132     runFirst();
133 133 };
134 134
135 135 /* --- server call (contract-frozen) --- */

```

```

136 136 function query(file, m, burst) {
137 137   const q = new URLSearchParams({ name: file.name });
138 138   if (m.fs) q.set("fs", m.fs);
139 139   if (m.fmt) q.set("fmt", m.fmt);
140 140   if (m.dtype) q.set("dtype", m.dtype);
141 141   if (m.endian) q.set("endian", m.endian);
142 142   if (m.bitrev) q.set("bitrev", "1");
143 143   if (m.rf != null) q.set("rf", m.rf);
144 144   if (burst != null) q.set("burst", burst);
145 145   return q;
146 146 }
147 147 async function postTo(api, file, m, burst) {
148 148   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
149 149   const data = await res.json();
150 150   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
151 151   return data;
152 152 }
153 153 async function runFirst() {                                     // entry: first analyze of a fresh capture
154 154   hideAll();
155 155   $("loadMsg").textContent = "신호를 분석하고 있어요...";
156 156   show($("loading"));
157 157   try {
158 158     const resp = await postTo(API, state.file, state.meta);
159 159     state.overview = resp.overview || null;
160 160     state.resp = resp;
161 161     render(resp);
162 162     if (state.overview) renderBurstMap(state.overview, resp);
163 163   } catch (e) {
164 164     showError(e.message);
165 165   }
166 166 }
167 167 async function analyze() {                                     // burst re-selection within a single signal
168 168   hideAll();
169 169   $("loadMsg").textContent = "신호를 분석하고 있어요...";
170 170   show($("loading"));
171 171   try {
172 172     state.resp = await postTo(API, state.file, state.meta, state.burst);
173 173     render(state.resp);
174 174     if (state.overview) renderBurstMap(state.overview, state.resp);
175 175     $("backBatch").classList.toggle("hidden", !state.batch.length);
176 176   } catch (e) {
177 177     showError(e.message);
178 178   }
179 179 }
180 180
181 181 /* --- batch mode --- */
182 182 async function runBatch(data, metas) {
183 183   hideAll();
184 184   show($("loading"));
185 185   const truths = new Map(metas.filter((m) => /\json$/i.test(m.name))
186 186     .map((m) => [m.name.replace(/\json$/i, ""), m]));
187 187   const sigmfs = new Map(metas.filter((m) => /\sigmf-meta$/i.test(m.name))
188 188     .map((m) => [m.name.replace(/\sigmf-meta$/i, ""), m]));
189 189   const rows = [];
190 190   for (let i = 0; i < data.length; i++) {
191 191     const f = data[i];
192 192     $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
193 193     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
194 194     // beats filename tokens
195 195     const sm = sigmfs.get(f.name.replace(/\.[^.]*/, ""));

```

```

196 196     const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
197 197     let resp = null, err = null;
198 198     try {
199 199         if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
200 200         resp = await postTo(API, f, meta);
201 201     } catch (e) { err = e.message; }
202 202     const tf = truths.get(f.name);
203 203     // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
204 204     // it used to read state.meta, which a fresh batch never set (crash) and a previous
205 205     // single-file run left stale (silent wrong-fs decode)
206 206     rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
207 207 }
208 208 state.batch = rows;
209 209 hideAll();
210 210 renderBatch(rows);
211 211 }
212 212
213 213 function renderBatch(rows) {
214 214     $("batchBody").innerHTML = rows.map((r, i) => {
215 215         if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
216 216             `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
217 217         const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
218 218         const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
219 219         return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
220 220             `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
221 221             `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
222 222     }).join("");
223 223     $("batchBody").querySelectorAll("tr").forEach((tr) => {
224 224         tr.onclick = () => {
225 225             const r = state.batch[+tr.dataset.i];
226 226             if (!r.resp) return;
227 227             state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
228 228             state.burst = null; state.overview = null;
229 229             hideAll();
230 230             render(r.resp);
231 231             const b = $("backBatch");
232 232             b.textContent = "← 목록으로";
233 233             b.onclick = () => { hideAll(); renderBatch(state.batch); };
234 234             b.classList.remove("hidden");
235 235         };
236 236     });
237 237     show($("batchCard"));
238 238 }
239 239 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
240 240 $("dlCsv").onclick = () => {
241 241     const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
242 242     const body = state.batch.filter((r) => r.resp).map((r) => {
243 243         const d = r.resp.detected, q = r.resp.quality;
244 244         return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
245 245             r.resp.snr_est_db ?? ""].join(",");
246 246     }).join("\n");
247 247     saveBlob("signus_batch.csv", head + body, "text/csv");
248 248 };
249 249
250 250 /* --- burst map: whole-record spectrogram strip with clickable time-burst boxes --- */
251 251 function renderBurstMap(ov, result) {
252 252     show($("burstMap"));
253 253     const n = ov.n || 1, bs = result.bursts || [];
254 254     paintStrip($("burstFall"), ov.strip);
255 255     const host = $("burstBoxes");

```

```

256 256 host.innerHTML = "";
257 257 bs.forEach((b, i) => {
258 258   const left = Math.max(0, b.start / n * 100);
259 259   const dur = (b.end - b.start) / ov.fs;
260 260   const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0
    ↳ ↳   ) + " ms"}`;
261 261   const box = document.createElement("button"); // full height (one signal), spans t0..t1 i
    ↳ ↳   n time
262 262   box.className = "box" + (i === result.burst_idx ? " sel" : "");
263 263   box.style.top = "0%"; box.style.height = "100%"; box.style.left = left + "%";
264 264   box.style.width = Math.min(100 - left, Math.max(0.8, (b.end - b.start) / n * 100)) + "%";
265 265   box.title = lbl;
266 266   box.innerHTML = `<span class="box-num">${i + 1}</span><span class="box-lane"></span>` +
267 267   `<span class="box-lbl">${lbl}</span>`;
268 268   box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persist
    ↳ ↳   s
269 269   host.appendChild(box);
270 270 });
271 271 }
272 272
273 273 /* --- rendering --- */
274 274 function statusOf(lock) {
275 275   if (lock >= 60) return ["복조 성공", "ok"];
276 276   if (lock >= 40) return ["부분 복조", "warn"];
277 277   return ["복조 실패", "bad"];
278 278 }
279 279 function render(d) {
280 280   hideAll();
281 281   show($("#results"));
282 282   $(".burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst sin
    ↳ ↳   gle mode
283 283   const lock = d.quality.lock;
284 284   let [txt, cls] = statusOf(lock);
285 285   if (d.detected.chirp) { txt = "처프 특성 보고"; cls = "warn"; }
286 286   const pill = $(".statusPill");
287 287   pill.textContent = txt;
288 288   pill.className = "pill " + cls;
289 289   $(".fileName").textContent =
290 290     `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
291 291   $(".merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB"
    ↳ ↳   ;
292 292   $(".evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + " %"
    ↳ ↳   ;
293 293   $(".snrVal").textContent = snrText(d);
294 294
295 295   const flags = [];
296 296   if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
297 297   if (d.family === "chirp") flags.push('<span class="chip warn">처프/CSS · 특성 판독</span>');
298 298   if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
299 299     d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
300 300   if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>
    ↳ ↳   ');
301 301   if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>
    ↳ ↳   >');
302 302   if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
    ↳ ↳   싱 가능</span>');
303 303   $(".flagRow").innerHTML = flags.join("");
304 304
305 305   renderBursts(d);
306 306   animateGauge(lock, cls);

```

```

307 307   renderParams(d);
308 308   drawSpectrum(d);
309 309   drawWaterfall(d);
310 310   startPlay(d);
311 311 }
312 312 function snrText(d) {
313 313   const s = d.snr_est_db;
314 314   if (s == null || d.quality.lock < 30) return "-";
315 315   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
316 316 }
317 317
318 318 function renderBursts(d) {
319 319   const row = $("burstRow"), bs = d.bursts || [];
320 320   // when the burst MAP is shown (multi-burst capture) the map replaces the chips
321 321   if (state.overview || bs.length < 2) { row.innerHTML = ""; return; }
322 322   const dur = (b) => {
323 323     const sec = (b.end - b.start) / d.fs;
324 324     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
325 325   };
326 326   row.innerHTML = bs.map((b, i) =>
327 327     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
328 328     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
329 329   row.querySelectorAll("button").forEach((el) => {
330 330     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
331 331   });
332 332 }
333 333
334 334 function chip(hit, truthTxt) {
335 335   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</span>`;
336 336 }
337 337 function renderParams(d) {
338 338   const det = d.detected, t = state.sidecar && state.sidecar.truth;
339 339   const near = (a, b, tol) => Math.abs(a - b) <= tol;
340 340   if (det.chirp) { // 처프/CSS: 성상도 제원 대신 특성 (복조 없음, 판독만)
341 341     const c = det.chirp;
342 342     const crows = [
343 343       ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc), null,
344 344         det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
345 345       ["처프", `${c.up ? "▲ 상승" : "▼ 하강"} · ${(c.mu / 1e9).toFixed(3)} MHz/ms`, null,
346 346         `점유대역폭 ${fmtHz(c.bw)}`],
347 347       ["변조방식", c.sf ? `LoRa 추정 SF${c.sf}` : "처프(FMCW/CSS)", null,
348 348         c.sf ? `심볼레이트 ${fmtHz(c.rs)} · 심볼시간 ${(c.tsym * 1e3).toFixed(2)} ms` : ""],
349 349     ];
350 350     $("paramGrid").innerHTML = crows.map(paramCard).join("");
351 351     return;
352 352   }
353 353   const rows = [
354 354     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
355 355       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
356 356       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
357 357     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.baud), fmtHz(t.baud)),
358 358       det.baud_conf ? `스펙트럼 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
359 359     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUpperCase(), t.mod.toUpperCase()),
360 360       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
361 361   ];
362 362   if (d.family === "fsk") {
363 363     rows.push(["변조지수 h", det.h.toFixed(2),

```



```

364 364         t && t.h !== null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), "");
365 365     } else {
366 366         rows.push(["롤오프", det.rolloff.toFixed(2),
367 367             t && t.rolloff !== null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2))
368 368             , ""]);
369 369     }
370 370     $("paramGrid").innerHTML = rows.map(paramCard).join("");
371 371 }
372 372
373 373 function paramCard([label, val, hit, note]) {
374 374     return `
375 375         <div class="card param">
376 376             <div class="param-label">${label}</div>
377 377             <div class="param-val">${val}</div>
378 378             ${note ? `<div class="param-note">${note}</div>` : ""}
379 379             ${hit ? `<div class="chips">${hit}</div>` : ""}
380 380         </div>`;
381 381     }
382 382
383 383 function animateGauge(lock, cls) {
384 384     const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
385 385     arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
386 386     arc.style.strokeDasharray = C;
387 387     const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
388 388     if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return
389 389         ; }
390 390     arc.style.strokeDashoffset = C;
391 391     requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
392 392     const t0 = performance.now();
393 393     const step = (t) => {
394 394         const k = Math.min(1, (t - t0) / 900);
395 395         num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
396 396         if (k < 1) requestAnimationFrame(step);
397 397     };
398 398     requestAnimationFrame(step);
399 399 }
400 400
401 401 /* --- canvas helpers --- */
402 402 function fit(cv, hCss) {
403 403     const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
404 404     const h = hCss || w;
405 405     cv.width = w * dpr; cv.height = h * dpr;
406 406     const g = cv.getContext("2d");
407 407     g.setTransform(dpr, 0, 0, dpr, 0, 0);
408 408     return [g, w, h];
409 409 }
410 410 function pct(a, p) {
411 411     const s = [...a].sort((x, y) => x - y);
412 412     return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
413 413 }
414 414
415 415 /* --- spectrum --- */
416 416 function drawSpectrum(d) {
417 417     // sa 프로브의 power spectrum 판과 같은 조판: 흑배경, 백색 곡선, 회색(0.35) 점선 격자,
418 418     // 바닥 p5-2 .. 최대+2 dB. 검출된 중심주파수(실선)와 ±baud/2(점선) 만 기능선으로 엮는다.
419 419     const [g, w, h] = fit($("#specCanvas"), 120);
420 420     g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
421 421     if (!d.spectrum) return;
422 422     const f = d.spectrum.f, db = d.spectrum.db;
423 423     const lo = pct(db, 0.05) - 2, hi = pct(db, 1) + 2;

```

```

422 422   const fmin = f[0], fmax = f[f.length - 1];
423 423   const X = (v) => (v - fmin) / (fmax - fmin) * w;
424 424   const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
425 425   g.strokeStyle = "rgba(255,255,255,.35)"; g.setLineDash([1, 5]);
426 426   for (let k = 1; k <= 4; k++) { // fs/2 의 k/5 지점 세로 격자 (sa 와 동일)
427 427     const gx = fmin + (fmax - fmin) * k / 5;
428 428     g.beginPath(); g.moveTo(X(gx), 0); g.lineTo(X(gx), h); g.stroke();
429 429   }
430 +   const det = d.detected, fca = Math.abs(det.fc) / 1e3, half = det.baud / 2e3;
431 431   g.setLineDash([4, 4]);
432 +   for (const gf of [fca - half, fca + half]) { // 단축(0..fs/2) 축 -- fc 는 접힌 절대값
433 433     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
434 434   }
435 435   g.setLineDash([]);
436 436   g.strokeStyle = "rgba(255,255,255,.6)";
437 +   g.beginPath(); g.moveTo(X(fca), 0); g.lineTo(X(fca), h); g.stroke();
438 438   g.strokeStyle = "#ebebeb"; g.lineWidth = 1.2; g.beginPath();
439 439   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
440 440   g.stroke();
441 441   g.fillStyle = "rgba(255,255,255,.6)"; g.font = "10px sans-serif";
442 442   g.fillText(sig(fmin) + " kHz", 4, h - 4);
443 443   const tmax = sig(fmax) + " kHz";
444 444   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
445 445 }
446 446
447 447 /* --- spectrogram strip (sa 프로브의 PNG 와 같은 조판) --- */
448 448 function drawWaterfall(d) { paintStrip($("#fallCanvas"), d.strip); }
449 449 function paintStrip(canvas, st) {
450 450   // 서버가 sa 와 같은 규칙(흑백 0..235, hamming 256/128, p25+10..35dB, 열 최대 폴링)으로
451 451   // 만든 격자를 해상도 그대로 찍는다. CSS 의 image-rendering: pixelated 가 sa PNG 를
452 452   // 확대해 보는 것과 같은 방식으로 늘린다 -- 보간 뭉개짐이 없다.
453 453   const g = canvas.getContext("2d");
454 454   canvas.width = st && st.cols ? st.cols : 1;
455 455   canvas.height = st && st.rows ? st.rows : 1;
456 456   if (!st || !st.cols) { g.fillStyle = "#000"; g.fillRect(0, 0, 1, 1); return; }
457 457   const img = g.createImageData(st.cols, st.rows);
458 458   for (let k = 0; k < st.g.length; k++) {
459 459     const o = k * 4;
460 460     img.data[o] = img.data[o + 1] = img.data[o + 2] = st.g[k];
461 461     img.data[o + 3] = 255;
462 462   }
463 463   g.putImageData(img, 0, 0);
464 464 }
465 465
466 466 /* --- constellation playback (the headline) --- */
467 467 const play = { raf: 0, i: 0, on: false, data: null };
468 468 let CG = null; // cached constellation geometry
469 469
470 470 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
471 471 function setPlayBtn(on) { $("#playBtn").textContent = on ? "⏏ 일시정지" : "▶ 재생"; }
472 472
473 473 function startPlay(d) {
474 474   stopPlay();
475 475   play.data = d;
476 476   play.i = 0;
477 477   const n = d.constellation.i.length;
478 478   $("scrub").max = String(Math.max(1, n));
479 479   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
480 480   drawConstBase(d);
481 481   if (REDUCED) { // static full view, no motion

```

```

482 482     drawPoints(d, 0, n);
483 483     $("scrub").value = String(n);
484 484     setPlayBtn(false);
485 485     return;
486 486 }
487 487 play.on = true;
488 488 setPlayBtn(true);
489 489 play.raf = requestAnimationFrame(loop);
490 490 }
491 491 $("playBtn").onclick = () => {
492 492     if (!play.data) return;
493 493     if (play.on) { stopPlay(); setPlayBtn(false); }
494 494     else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
495 495 };
496 496 $("scrub").oninput = () => {
497 497     if (!play.data) return;
498 498     stopPlay(); setPlayBtn(false);
499 499     play.i = +$("scrub").value;
500 500     drawConstBase(play.data);
501 501     drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
502 502 };
503 503
504 504 function loop() {
505 505     const d = play.data, n = d.constellation.i.length;
506 506     const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
507 507     const to = Math.min(n, play.i + step);
508 508     fade();                                // phosphor persistence
509 509     drawPoints(d, play.i, to);
510 510     play.i = to >= n ? 0 : to;
511 511     if (play.i === 0) drawConstBase(d);    // loop: clear the trail
512 512     $("scrub").value = String(play.i);
513 513     if (play.on) play.raf = requestAnimationFrame(loop);
514 514 }
515 515
516 516 function drawConstBase(d) {
517 517     const [g, w] = fit($("#constCanvas"));
518 518     const fsk = d.family === "fsk";
519 519     const I = d.constellation.i, Q = d.constellation.q;
520 520     const ideal = fsk ? [] : idealPoints(d.detected.mod);
521 521     let lim = 0;
522 522     for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
523 523     for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
524 524     lim = (lim || 1) * 1.12;
525 525     const R = w / 2 * 0.9;
526 526     CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
527 527
528 528     g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
529 529     g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
530 530     for (let f = -1; f <= 1; f += 0.5) {
531 531         const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
532 532         g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
533 533         g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
534 534     }
535 535     if (fsk) {                                // frequency levels as dashed bands
536 536         g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
537 537         for (const lv of [...new Set(I.map((v) => Math.round(v)))] .filter((v) => Math.abs(v) <= 8)
538 538             ) {
539 539             const [, yy] = CG.map(0, lv);
540 540             g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();

```

```

541 541     g.setLineDash([]);
542 542 } else {
543 543     g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
544 544     for (const p of ideal) {
545 545         const [x, y] = CG.map(p.i, p.q);
546 546         g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
547 547     }
548 548 }
549 549 }
550 550 function fade() {
551 551     if (!CG) return;
552 552     CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
553 553     CG.g.fillRect(0, 0, CG.w, CG.w);
554 554 }
555 555 function drawPoints(d, from, to) {
556 556     if (!CG || to <= from) return;
557 557     const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
558 558     const fsk = d.family === "fsk";
559 559     const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
560 560     g.globalCompositeOperation = "lighter";
561 561     g.fillStyle = `rgba(90,170,255,${a})`;
562 562     for (let k = from; k < to; k++) {
563 563         // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
564 564         const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
565 565         g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
566 566     }
567 567     g.globalCompositeOperation = "source-over";
568 568 }
569 569
570 570 /* --- downloads --- */
571 571 function saveBlob(name, text, type) {
572 572     const url = URL.createObjectURL(new Blob([text], { type }));
573 573     const a = document.createElement("a");
574 574     a.href = url; a.download = name; a.click();
575 575     URL.revokeObjectURL(url);
576 576 }
577 577 const stem = () => state.file.name.replace(/\.([^.]*)$/, "");
578 578 $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
579 579     const btn = e.currentTarget, was = btn.textContent;
580 580     try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
581 581     catch { btn.textContent = "복사 실패"; }
582 582     setTimeout(() => { btn.textContent = was; }, 1400);
583 583 };
584 584 $("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
585 585 $("dlSyms").onclick = () => {
586 586     const c = state.resp.constellation;
587 587     saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/csv");
588 588 };
589 589 $("dlReport").onclick = () =>
590 590     saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");
591 591
592 592 /* --- view helpers --- */
593 593 function show(el) { el.classList.remove("hidden"); }
594 594 function hideAll() {
595 595     stopPlay();
596 596     ["metaForm", "loading", "errorCard", "results", "batchCard"]
597 597     .forEach((id) => $(id).classList.add("hidden"));
598 598     $("backBatch").classList.add("hidden");
599 599 }

```

```
600 600 function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
601 601
602 602 /* --- dropzone wiring --- */
603 603 const drop = $("dropCard"), input = $("fileInput");
604 604 // reset value first: re-picking the SAME file fires no change event otherwise
605 605 const pick = () => { input.value = ""; input.click(); };
606 606 drop.onclick = pick;
607 607 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
    ↳ ↳ ; } };
608 608 input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
609 609 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
610 610 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
611 611 drop.addEventListener("drop", (e) => {
612 612   e.preventDefault(); drop.classList.remove("drag");
613 613   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
614 614 });
```